Universidade de Brasília
Departamento de Ciência da Computação
Bacharelado em Ciência da Computação

Projeto e Simulação de uma ULA RISCV

Aluno: Lucas Vasconcelos Pirani Carneiro
Matrícula: 251031785

1 Introdução

A Unidade Lógica e Aritmética (ULA) é um componente combinacional essencial do caminho de dados de um processador, responsável pela execução de operações aritméticas e lógicas cujas saídas dependem exclusivamente dos valores presentes em suas entradas [2].

No contexto da arquitetura RISC-V, a ULA é utilizada em operações de processamento de dados, no cálculo de endereços de memória e na avaliação de condições para desvios, conforme os sinais de controle recebidos [2].

Diante disso, este trabalho tem como objetivo geral projetar e simular uma ULA de 32 bits, de acordo com as operações definidas na especificação do projeto [1]. Para alcançar esse objetivo, foram estabelecidos os seguintes objetivos específicos: implementar operações aritméticas, lógicas, de deslocamento e de comparação; produzir as saídas `Z` e `cond` de acordo com a operação selecionada; e verificar o funcionamento do circuito por meio de um *testbench* desenvolvido em VHDL.

2 Implementação

2.1 Interface

O módulo `ulaRV` recebe dois operandos, `A` e `B`, e o código da operação, `opcode`. A largura dos operandos e da saída pode ser definida pelo genérico `WSIZE`, cujo valor padrão é 32 bits. A saída `Z` apresenta o resultado da operação, enquanto `cond` indica o resultado das operações de comparação: vale 1 quando a condição é verdadeira e 0 quando é falsa ou quando a operação não é uma comparação [1].

```
entity ulaRV is
    generic (WSIZE : natural := 32);
    port (
        opcode : in  std_logic_vector(3 downto 0);
        A, B   : in  std_logic_vector(WSIZE - 1 downto 0);
        Z      : out std_logic_vector(WSIZE - 1 downto 0);
        cond   : out std_logic
    );
end ulaRV;
```

Nas operações de comparação, o resultado também é colocado no bit menos significativo de `Z`: a saída recebe o valor 1 quando a condição é verdadeira e 0 quando é falsa [1]. O sinal `cond` disponibiliza essa condição para o controle do processador, que pode combiná-la com o sinal `Branch` para decidir se um desvio condicional deve ocorrer [1, 2].

2.2 Arquitetura

A arquitetura `behavioral` implementa as operações em um processo combinacional, selecionadas pelo código de quatro bits de `opcode`. Os códigos utilizados são os definidos na especificação.

As operações `ADD` e `SUB` convertem os operandos para o tipo `signed` antes de realizar a soma ou a subtração. O resultado é convertido novamente para `std_logic_vector`. As operações `AND`, `OR` e `XOR` são realizadas bit a bit.

As operações de deslocamento usam as funções `shift_left` e `shift_right` da biblioteca `numeric_std`. Em `SLL`, o operando é tratado como `unsigned`; em `SRL`, o deslocamento à direita preenche os bits mais significativos com zeros; em `SRA`, o operando é tratado como `signed`, preservando o bit de sinal. A quantidade de posições deslocadas é obtida dos cinco bits menos significativos de `B`, pois a quantidade máxima de bits que é possível deslocar são 32.

Nas comparações `SLT` e `SGE`, os operandos são interpretados com sinal. Em `SLTU` e `SGEU`, são interpretados sem sinal. As operações `SEQ` e `SNE` verificam, respectivamente, se os operandos são iguais ou diferentes. Quando uma comparação é verdadeira, `Z` recebe o valor 1 e `cond` também recebe 1; quando é falsa, ambos indicam zero. Para as demais operações, `cond` permanece em zero.

O código completo da ULA está disponível no arquivo `ulaRV.vhd`.

3 Testes e Testbench

O testbench instancia o módulo `ulaRV`, aplica valores aos operandos `A` e `B` e seleciona a operação por meio de `opcode`. Após cada estímulo, aguarda-se 1 ns para que as saídas combinacionais se estabilizem. Em seguida, instruções `assert` comparam os valores de `Z` e `cond` com os resultados esperados.

Foram incluídos casos para todos os códigos de operação implementados. As operações aritméticas foram testadas com resultados positivos, negativos e iguais a zero, conforme solicitado na especificação. Também foram testados deslocamentos e comparações verdadeiras e falsas. Para as comparações com e sem sinal, foram escolhidos operandos que permitem observar a diferença entre as duas interpretações.

O código do testbench está disponível no arquivo `ulaRV_tb.vhd`.

3.1 Simulação

A simulação foi executada na plataforma EDA Playground com o simulador `GHDL 5.1.1`. Ao final da execução, o testbench apresentou a mensagem de conclusão dos testes e não foram observadas mensagens de falha das asserções.

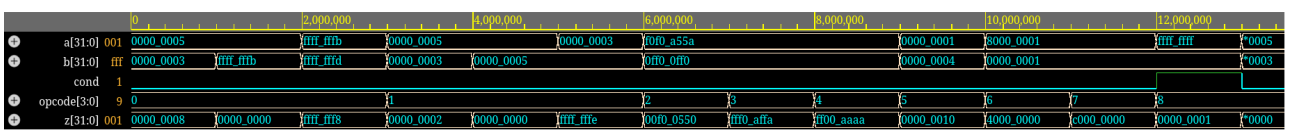


Figura 1: Formas de onda da primeira parte.

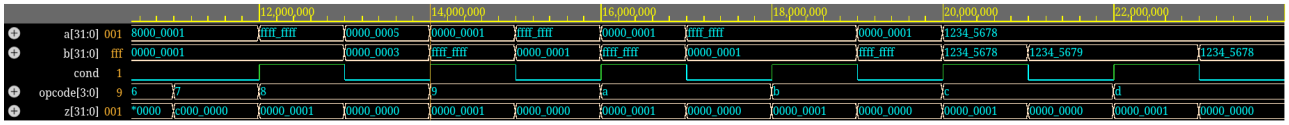


Figura 2: Formas de onda da segunda parte.

4 Discussão

4.1 Diferença entre as comparações com e sem sinal

Nas comparações com sinais os bits são tratados como inteiros em complemento de dois, sendo assim se o bit mais significativo for 1 o valor é negativo e, portanto, menor que qualquer valor positivo. Enquanto, nas comparações sem sinal, o mesmo padrão de bits é interpretado como um inteiro não negativo entre fazendo com que um bit mais significativo igual a 1 represente um número muito grande, maior que qualquer valor iniciado por 0 [2].

4.2 Como detectar overflow nas operações ADD e SUB

O *overflow* ocorre quando o resultado de uma operação aritmética não pode ser representado no intervalo de valores permitido pela quantidade de bits utilizada.

Na operação de adição (`ADD`), ocorre *overflow* quando os operandos possuem o mesmo sinal, mas o resultado apresenta sinal diferente. Na subtração (`SUB`), o *overflow* ocorre quando os operandos possuem sinais diferentes e o resultado apresenta sinal diferente do minuendo [2].

Nas operações sem sinal, a detecção segue critérios diferentes. Na adição, o *overflow* ocorre quando há um transporte (*carry-out*) para além do bit mais significativo, que não pode ser armazenado no resultado de 32 bits. Na subtração, ocorre um empréstimo (*borrow*) quando o minuendo é menor que o subtraendo [2].

Na arquitetura RISC-V, as instruções inteiras convencionais, como `ADD` e `SUB`, não atualizam automaticamente indicadores de condição nem geram exceções específicas de *overflow* aritmético [2].

Dessa forma, essa detecção deve ser implementada explicitamente, por meio da análise dos operandos e do resultado ou de operações adicionais de comparação. Na implementação da ULA, a detecção poderia ser incorporada ao circuito mediante a inclusão de uma saída específica para indicar a ocorrência de *overflow*, sem alterar necessariamente a saída de dados já existente [2].

5 Conclusão

Este trabalho apresentou o projeto e a simulação de uma ULA de 32 bits para a arquitetura RISC-V, implementada em VHDL. O módulo desenvolvido realiza operações aritméticas, lógicas, de deslocamento e de comparação, gerando corretamente as saídas `Z` e `cond`. A validação por meio do *testbench* confirmou o funcionamento esperado para todos os códigos de

operação, incluindo casos com resultados positivos, negativos, nulos e comparações com e sem sinal. Assim, os objetivos propostos foram atendidos, demonstrando a adequação da ULA para integração ao caminho de dados de um processador RISC-V.

Referências

[1] Especificação do trabalho: *Projeto e Simulação de uma ULA RISCV em VHDL*. Material fornecido pelo(a) professor(a), s.d.

[2] PATTERSON, David A.; HENNESSY, John L. *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*. 2. ed. Morgan Kaufmann, 2020. ISBN: 978-0-12-820331-6.